

AutoRestTest: A Tool for Automated REST API Testing Using LLMs and MARL

Tyler Stennett^{*1}, Myeongsoo Kim^{*2}, Saurabh Sinha^{†3}, Alessandro Orso^{*4}

^{*}Georgia Institute of Technology, Atlanta, Georgia, USA

Email: {tyler.stennett¹, mkim754²}@gatech.edu, orso@cc.gatech.edu⁴

[†]IBM Research, Yorktown Heights, New York, USA

Email: sinhas@us.ibm.com³

Abstract—As REST APIs have become widespread in modern web services, comprehensive testing of these APIs is increasingly crucial. Because of the vast search space of operations, parameters, and parameter values, along with their dependencies and constraints, current testing tools often achieve low code coverage, resulting in suboptimal fault detection. To address this limitation, we present AutoRestTest, a novel tool that integrates the Semantic Property Dependency Graph (SPDG) with Multi-Agent Reinforcement Learning (MARL) and large language models (LLMs) for effective REST API testing. AutoRestTest determines operation-dependent parameters using the SPDG and employs five specialized agents (operation, parameter, value, dependency, and header) to identify dependencies of operations and generate operation sequences, parameter combinations, and values. Through an intuitive command-line interface, users can easily configure and monitor tests with successful operation count, unique server errors detected, and time elapsed. Upon completion, AutoRestTest generates a detailed report highlighting errors detected and operations exercised. In this paper, we introduce our tool and present preliminary findings, with a demonstration video available at <https://www.youtube.com/watch?v=VVus2W8rap8>.

Index Terms—Multi Agent Reinforcement Learning for Testing, Automated REST API Testing

I. INTRODUCTION

Representational State Transfer (REST) APIs serve as the backbone of modern web services, with nearly 90% of developers engaging with APIs and approximately 86% of these APIs employing the REST architecture. By emphasizing a lightweight design and scalability, REST APIs seamlessly connect software systems through standard web protocols, such as the HyperText Transfer Protocol (HTTP). This approach supports a client-server architecture that effectively separates responsibilities while enhancing the efficiency and maintainability of web services [1].

The critical role of REST APIs in web service interactions has sparked significant interest in automated techniques for testing these APIs, particularly following the introduction of the Open API Specification (OAS) [2], and many researchers and practitioners have proposed a variety of techniques in this space.

For instance, RESTler employs search algorithms (BFS, DFS, and Random Walk), while EvoMaster [3] utilizes evolutionary algorithms. For another example, MoRest [4] manages operation dependencies through a dynamically updated RESTful Property Graph (RPG) derived from the OAS. Re-

inforcement learning has also been applied in this domain, with ARAT-RL [5] and DeepREST [6] optimizing parameter selection and search space exploration. More recently, large language models have been leveraged to further enhance testing: LlamaRestTest [7] focuses on value generation and inter-parameter dependencies using language models, while NLP2REST [8] and RESTGPT [9] enhance the OAS by extracting actionable rules from natural language descriptions.

While existing research has addressed individual challenges of REST API testing, such as parameter generation, inter-parameter dependencies, and operation dependencies, no comprehensive technique has yet tackled these challenges together effectively. To address this limitation, in this paper we introduce AutoRestTest, a tool that combines graph-based dependency modeling, Large Language Models (LLMs), and Multi-Agent Reinforcement Learning (MARL) to perform REST API testing holistically. Key features of AutoRestTest include:

- The **Semantic Property Dependency Graph (SPDG)** enables the use of a dependency agent to reduce the search space and explore API dependencies efficiently.
- **REST Agents** effectively identify headers, operations, parameter combinations, and their corresponding values.
- An **LLM** model creates realistic inputs for both value and header generation.
- A friendly **User Interface** provides a command line interface (CLI) alongside detailed reports that highlight the results of the tool, including detected internal server errors.

II. AUTORESTTEST

Figure 1 illustrates the AutoRestTest architecture, which consists of three primary modules: the SPDG, the REST agents, and the request generator. The process begins by parsing the OAS to extract endpoints as well as their request/response schemas. Using this information, the dependency agent constructs the SPDG, representing API operations as nodes and semantic similarities between operation inputs and outputs as edges. Once the graph is constructed, the operation, dependency, value, parameter, and header agents are initialized with zeroed policy tables using information from the SPDG. These tables are subsequently optimized using Q-learning [10] to identify optimal input combinations

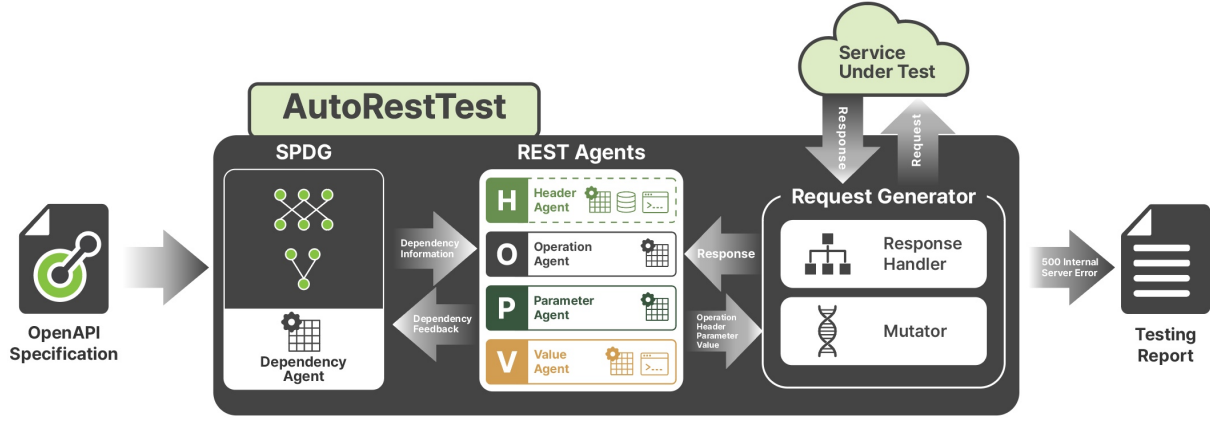


Fig. 1. Overview of AutoRestTest.

for generating realistic requests for each endpoint. In this section, we provide a brief overview of each component; a detailed discussion of our approach is available in our research paper [11].

A. Semantic Property Dependency Graph

The SPDG is constructed by parsing the input OAS and assigning each API operation as a vertex. AutoRestTest then iterates through each pair of vertices, using a lightweight GloVe word-embedding model [12] to measure semantic similarity between the parameters, request body, and responses of the two operations corresponding to the vertices. An edge is added between vertices if the similarity value between any two items exceeds a predefined threshold (0.7 in our current implementation). For operations with no dependencies above this threshold, the top three highest similarity matches are added to the graph.

The dependency agent manages the SPDG during request generation. To do so, it communicates with the value agent to use stored parameters, request bodies, and responses from successful requests in future queries, validating the dependencies identified in the SPDG.

B. REST Agents

The REST agents consist of four specialized components, each addressing a distinct aspect of the request generation process. First, the Operation Agent selects the operation for the request. Next, the Parameter Agent determines which parameters should be included. Then, the Value Agent identifies the data source and assigns values to the selected parameters, drawing from the LLM, the dependency agent, or the default settings. Lastly, the Header Agent leverages account-related operations from the specification to supply basic token authentication headers.¹

C. Q-Learning

Both the SPDG and REST agent modules use Q-learning to determine the optimal actions for the operation, parameter, value, dependency, and header agents. This process involves Q-table initialization, action selection, and reward delegation.

¹The header agent is a recent addition not described in our research paper.

1) **Q-Table Initialization:** A Q-table is a data structure that maps action options to expected cumulative rewards. For example, the parameter agent's Q-table lists all combinations of an operation's parameters and request body properties as potential actions, with each value initialized to zero until updated through learning.

2) **Action Selection:** Agents select actions using an epsilon-greedy strategy [10] that balances exploration and exploitation. The exploration probability ($f(\epsilon) = 1 - \epsilon$) with epsilon-decay allows sufficient exploration in the early stages.

3) **Reward Delegation:** After an action, AutoRestTest's Response Handler updates the Q-table using the Bellman equation [10]. The update process involves retrieving the current Q-value, calculating the new Q-value, and updating the Q-table. Rewards are assigned as follows:

1. The operation agent is rewarded for finding client (4xx) and server (5xx) errors.
2. The other agents (value, parameter, dependency, and header) are rewarded for successes (2xx).

D. Request Generator

The Request Generator constructs and dispatches requests using data from the previous modules. It operates in a defined sequence of steps consisting of communication, modification, and response handling. During communication, it interacts with the REST agents to determine the selected operation and the assigned parameter and header values. The modification step employs a custom mutator to randomly modify requests, potentially exposing additional server errors. This mutator probabilistically selects from options such as parameter type alterations, name mutations, media type changes, random dependency selections, and token changes to generate new values of random length. Finally, the response handling step processes responses from completed requests to determine the reward for the exploring agent, thereby refining the AutoRestTest model.

III. TOOL USAGE

To use AutoRestTest, the user must first launch the Service Under Test (SUT), exposing a URL for interaction. Next, they should configure AutoRestTest according to the SUT's requirements. Users can then use AutoRestTest's CLI to run the tool, with output data being made accessible after completion.

A. Configuration

AutoRestTest offers various configuration options to tailor each module to the specific requirements of the SUT. A centralized **configurations.py** file in the root directory of the tool streamlines the process of adjusting these settings.

1) *Specification Selection*: To select the input specification corresponding to the SUT, users must note the document location relative to the root directory in the configuration file. AutoRestTest's custom parser accepts only OAS 3.0 formatted inputs. However, users can easily transfer their outdated Swagger 2.0 files using the public Swagger Converter. It is essential to ensure that the URL supplied in the specification matches the exposed URL of the SUT.

2) *Large Language Model Engine and Parameters*: To accommodate varying budgets and larger services with deeply nested objects that require extensive context windows, AutoRestTest allows users to select the Large Language Model (LLM) engine and configure parameters for its value agent from OpenAI's fleet. Specifically, AutoRestTest supports all recent models with built-in price tracking for **GPT-4o**, GPT-4o mini, o1, and o1-mini (for cost transparency).

The LLM temperature parameter, adjustable in AutoRestTest, influences output diversity and determinism. The default setting of 0.7 balances accuracy and creativity, with higher values (> 1) producing more diverse outputs and lower values (≈ 0) and yielding more deterministic results.

3) *Caching*: To avoid redundant use of the LLM, AutoRestTest incorporates optional local caching through Python's **shelve** object persistence library. While caching is enabled, the SPDG and LLM-generated Q-tables are stored in a database file. Subsequent executions attempt to use these cached values, significantly reducing testing costs. However, users should disable caching when making changes to the graph or Q-table to allow regeneration of the database files.

4) *Q-Learning Parameters*: AutoRestTest employs the value decomposition approach to Q-learning [13], which integrates a *learning rate* and a *discount factor* to ensure Q-table convergence. By default, AutoRestTest uses values 0.1 and 0.9 for the learning rate and discount factor, respectively [5]. Users can adjust these parameters to influence the convergence speed and agent behavior. Basically, a higher learning rate would result in more drastic updates of the Q-table values, potentially increasing convergence speed but lowering accuracy; a lower discount rate would diminish the importance of later requests, heavily emphasizing the initial queries.

5) *Request Generator Modifications*: Two configurable parameters govern the request generation process: the time duration and the mutation rate. Given the complexity of the agent design and the large Q-tables, AutoRestTest benefits from an extended execution period to ensure adequate request diversity and convergence. In addition, increasing the mutation rate broadens the range of generated requests, while potentially slowing Q-value convergence.

```
1 {
2   "Title": "Report for 'Api Documentation' (market2)",
3   "Duration": "300 seconds",
4   "Total Requests Sent": 3991,
5   "Status Code Distribution": {
6     "500": 149,
7     "401": 3509,
8     "200": 99,
9     "406": 215,
10    "404": 19
11  },
12  "Number of Total Operations": 13,
13  "Number of Successfully Processed Operations": 3,
14  "Percentage of Successfully Processed Operations": "
    23.08%",
15  "Number of Unique Server Errors": 104,
16  "Operations with Server Errors": {
17    "addItemUsingPUT": 31,
18    "createCustomerUsingPOST": 45,
19    "updateContactsUsingPUT": 21,
20    "getProductUsingGET": 21,
21    "getOrderUsingGET": 12,
22    "setDeliveryUsingPUT": 6,
23    "payByCardUsingPOST": 13
24  }
25 }
```

Listing 1. AutoRestTest Report Output for the Market Tool

B. Command Line Interface

Users can operate and interact with AutoRestTest through its CLI once the program has been configured. To begin, users should either install the requirements listed in the **requirements.txt** file or enable the Conda environment within the **auto-rest-test.yaml** file.

The CLI regularly updates users on AutoRestTest's progress. Major milestones include the creation of the SPDG, the instantiation of the Q-learning policy tables, and the commencement of the request generation process. Given the complexity of each step, the CLI provides intermediary messages between these milestones. If unexpected errors occur, such as issues with caching, AutoRestTest notifies the user through the CLI before handling the issue and continuing.

During the request generation and Q-learning phases, the CLI outputs information about operation coverage and time elapsed. This includes the number of **unique server errors** identified and successful operations processed, the distribution of status codes, and the percentage of time elapsed. This continuous output allows users to quickly assess AutoRestTest's efficiency and patterns in error identification over time.

C. Report Generation

Upon completion, AutoRestTest compiles comprehensive data from exercising the SUT into a sequence of files for user access and evaluation. These files are available in the **data/** folder of the specified root directory:

- **report.json** summarizes AutoRestTest's findings, including status code distributions, total successful operations, and **unique server errors**. Listing 1 shows an example report.
- **server_errors.json** stores details of every request that resulted in server errors (5xx) for reproducibility.
- **operation_status_codes.json** contains the distribution of status codes for each operation.
- **successful_parameters.json** lists parameter assignments that returned successful (2xx) responses by operation.

TABLE I

NUMBER OF OPERATIONS EXERCISED.

	AutoRestTest	ARAT-RL	EvoMaster	MoRest	RESTler
FDIC	6	6	6	6	6
OMDb	1	1	1	1	1
OhSome	12	0	0	0	0
Spotify	7	5	4	4	3
Total	26	12	11	11	10

- **successful_bodies.json** provides request body properties that returned successful (2xx) responses by operation.
- **successful_primitives.json** contains request bodies with no associated properties that returned successful (2xx) responses by operation.
- **q_tables.json** presents the converged Q-table values for each agent across all operations.

By evaluating these files, users can identify both strengths and weaknesses in a given API. The parameter agent’s Q-table indicates successful parameter combinations and inter-parameter dependencies. The dependency agent exposes relationships between parameters across operations. The operation status code distribution visualizes which operations are comprehensive and easy to process. Finally, analyzing server errors can help users improve the reliability of their service.

IV. PRELIMINARY RESULTS

We evaluated the performance of AutoRestTest alongside the four state-of-the-art REST API testing tools used in the ARAT-RL study: RESTler [14] (v9.2.4), EvoMaster [3] (v3.0.0), ARAT-RL [5] (v0.1), and MoRest [4] (obtained directly from the authors). All tools were tested against four real-world RESTful services included in a recent study [8], namely **FDIC**, **OMDb**, **OhSome**, and **Spotify**. To quantify effectiveness, we measured the number of successfully processed operations (2xx status codes) within a one-hour testing window, a preferred metric for comparing REST API testing tools [15]. As shown in **Table I**, our approach covered 26 unique operations across these services, outperforming ARAT-RL (12), EvoMaster (11), MoRest (11), and RESTler (10).

Notably, AutoRestTest was the only tool able to generate 2xx responses for the OhSome service, one of the most challenging RESTful services considered; while other tools were only able to trigger 4xx status codes, AutoRestTest successfully processed 12 operations in the service. Our tool was also the strongest performer on the Spotify service. Moreover, in an evaluation of internal server errors, AutoRestTest was the **only tool to detect a 5xx status code on the Spotify** service. We have reported the error and are awaiting a response from the developers. These results provide initial, yet clear evidence that AutoRestTest is effective in testing real-world RESTful APIs, including for complex services.

V. CONCLUSION

We introduced AutoRestTest, a new tool that combines a SPDG, LLMs, and MARL to effectively test REST APIs. We described our approach, demonstrating how AutoRestTest addresses key challenges in API testing through advanced

dependency modeling and intelligent request generation. Additionally, we conducted a preliminary study that shows the effectiveness of our tool in practice. We provided a practical demonstration of the tool’s usage and made the artifact available for further evaluation and replication [16].

ACKNOWLEDGMENTS

This work was partially supported by NSF, under grant CCF-0725202 and DOE, under contract DE-FOA-0002460, and gifts from Facebook, Google, IBM Research, and Microsoft Research.

REFERENCES

- [1] L. Richardson, M. Amundsen, and S. Ruby, *RESTful Web APIs: Services for a Changing World*. O’Reilly Media, Inc., 2013.
- [2] M. Kim, Q. Xin, S. Sinha, and A. Orso, “Automated test generation for rest apis: No time to rest yet,” in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 289–301.
- [3] A. Arcuri, “Restful api automated test case generation with evomaster,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 28, no. 1, jan 2019.
- [4] Y. Liu, Y. Li, G. Deng, Y. Liu, R. Wan, R. Wu, D. Ji, S. Xu, and M. Bao, “Morest: Model-based restful api testing with execution feedback,” in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1406–1417.
- [5] M. Kim, S. Sinha, and A. Orso, “Adaptive rest api testing with reinforcement learning,” in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Los Alamitos, CA, USA: IEEE Computer Society, sep 2023, pp. 446–458.
- [6] D. Corradini, Z. Montolli, M. Pasqua, and M. Ceccato, “Deeprest: Automated test case generation for rest apis exploiting deep reinforcement learning,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.08594>
- [7] M. Kim, S. Sinha, and A. Orso, “Llamaresttest: Effective rest api testing with small language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.08598>
- [8] M. Kim, D. Corradini, S. Sinha, A. Orso, M. Pasqua, R. Tzoref-Brill, and M. Ceccato, “Enhancing rest api testing with nlp techniques,” in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 1232–1243.
- [9] M. Kim, T. Stennett, D. Shah, S. Sinha, and A. Orso, “Leveraging large language models to improve rest api testing,” in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, ser. ICSE-NIER’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 37–41. [Online]. Available: <https://doi.org/10.1145/3639476.3639769>
- [10] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: A Bradford Book, 2018.
- [11] M. Kim, T. Stennett, S. Sinha, and A. Orso, “A multi-agent approach for rest api testing with semantic graphs and llm-driven inputs,” *arXiv preprint arXiv:2411.07098*, 2024.
- [12] J. Pennington, R. Socher, and C. D. Manning, “Glove: Global vectors for word representation,” in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 2014, pp. 1532–1543.
- [13] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, and T. Graepel, “Value-decomposition networks for cooperative multi-agent learning,” *arXiv preprint arXiv:1706.05296*, 2017.
- [14] V. Atlidakis, P. Godefroid, and M. Polishchuk, “Restler: Stateful rest api fuzzing,” in *Proceedings of the 41st International Conference on Software Engineering*, ser. ICSE ’19. Piscataway, NJ, USA: IEEE Press, 2019, p. 748–758.
- [15] A. Golmohammadi, M. Zhang, and A. Arcuri, “Testing restful apis: A survey,” *ACM Trans. Softw. Eng. Methodol.*, aug 2023.
- [16] SE@GT, “Experiment infrastructure, data, and results for autoresttest,” <https://github.com/selab-gatech/AutoRestTest>, 2024.